Processingをはじめよう

第10章

オブジェクト

補足説明

目次

- クラスとオブジェクト　補足
  - ブロックを作成
  - フィールドの追加
  - コンストラクタの作成とフィールドへの代入
  - メソッドの追加

Tutorials Objects

- processing.org Tutorials
  - Objects を使って補足説明

    https://processing.org/tutorials/objects/
    - 人間(Human)型オブジェクトを作るためクラスを定義
    - クラス名 Human

      クラスのブロックを作ると・・・

```
class Human {



}
```

クラスの
ブロックを作る

Tutorials Objects

- 人間型のオブジェクトは以下の情報を持っていると考え、データ型と名前を決めます
  - 身長　　float height
  - 体重　　float weight
  - 性別　　int　gender
  - 目の色　color eyeColor
  - 髪の色　color hairColor

```
class Human {
  float height;
  float weight;
  int gender;
  color eyeColor;
  color hairColor;
}
```

クラスの
フィールドを追加

- オブジェクトが持つ情報はクラスのフィールドとして定義

Tutorials Objects

- 人間型のオブジェクトは、一人ひとり違う情報（名前や身長体重など）を持っている
  - これらの情報をコンストラクタを使って初期値を設定する
    - コンストラクタの名前はクラスと同じ、戻り値の型はつけない
    - 初期値を設定するフィールドの数だけ引数を設定する

Tutorials Objects

```
class Human {
  float height;
  float weight;
  int gender;
  color eyeColor;
  color hairColor;

  Human(float h, float w, int g, color eye, color hair){
    height = h;
    weight = w;
    gender = g;
    eyeColor = eye;
    hairColor = hair;
  }

}
```

初期値を設定したい
フィールドが5つある

コンストラクタを追加

コンストラクタの引数を5つ設定する
引数型はフィールドと同じ型
引数名はフィールド名と
区別できるようにするとよい

フィールドに
引数（初期値）を代入する

Tutorials Objects

- 人間型のオブジェクトの持っている機能を以下のように考え、戻り値の型と名前を決めます
- メソッドを定義する
  - 眠る　`void sleep()`
  - 目を覚ます　`void wakeUp()`
  - 食べる　`void eat()`
  - 乗り物に乗る　`void ride()`

Tutorials Objects

```
class Human {
  float height;
  float weight;
  int gender;
  color eyeColor;
  color hairColor;

  Human(float h, float w, int g,
                color eye, color hair){
    height = h;
    weight = w;
    gender = g;
    eyeColor = eye;
    hairColor = hair;
  }

  void sleep(){
    println("zzz");
  }

  void wakeUp(){
    println("Good Morning");
  }

  void eat(){
    println("Yummy");
  }

  void ride(){
    println("Boom");
  }

}
```

4つの
メソッドを
追加

Tutorials Objects

- 人間型のオブジェクトaomoriを使ってプログラムを作る
  - オブジェクト変数の宣言
    クラス名　オブジェクト変数名;

```
Human aomori;
```

  - setup()でオブジェクトを生成する

```
void setup(){
  aomori = new Human(引数省略);
}
```

new演算子

コンストラクタ

コンストラクタの引数に初期値を設定する

  - メソッドを呼び出す　オブジェクト.メソッド();

```
aomori.sleep();
```

Tutorials Objects

```
Human aomori;

void setup(){
  aomori = new Human(170.0, 65.0, 0, color(0), color(0));
}

void draw(){
  aomori.wakeUp();
  aomori.eat();
  aomori.ride();
  aomori.sleep();
}
```

オブジェクト変数の宣言

オブジェクトの生成

メソッドを呼び出す

```
class Human {
  float height;
  float weight;
  int gender;
  color eyeColor;
  color hairColor;

  Human(float h, float w, int g, color eye, color hair){
    height = h;
    weight = w;
    gender = g;
    eyeColor = eye;
    hairColor = hair;
  }

  void sleep(){ println("zzz"); }
  void wakeUp(){ println("Good Morning"); }
  void eat(){ println("Yummy"); }
  void ride(){ println("Boom"); }

}
```

練習問題

- 犬オブジェクトを作るためDogクラスを定義
  - フィールド
    - 名前　`String name`
    - 身長　`float height`
    - 体重　`float weight`
    - 体毛の色　`color bodyColor;`
  - メソッド
    - 歩く　`void walk()`
    - 走る　`void run()`
    - ほえる　`void bark()`

練習問題

```
// 練習問題
オブジェクト変数の宣言

void setup(){
    オブジェクトの生成
}

void draw(){
    メソッド 歩く を呼び出す
    メソッド 走る を呼び出す
    メソッド ほえる を呼び出す
}
```

```
class Dog {
    フィールドの追加

    コンストラクタの追加と
    フィールドの代入

    メソッドの追加
}
```

Tutorials Objects Cars

```
color c;
float xpos;
float ypos;
float xspeed;

void setup(){
  size(200, 200);
  c = color(255);
  xpos = width/2;
  ypos = height/2;
  xspeed = 1;
}

void draw(){
  background(0);
  display();
  drive();
}

void display(){
  rectMode(CENTER);
  fill(c);
  rect(xpos, ypos, 20, 10);
}
void drive(){
  xpos = xpos + xspeed;
  if(xpos > width){
    xpos  0;
  }
}
```

```
class Car {
  color c;
  float xpos;
  float ypos;
  float xspeed;


  Car(){
    c = color(255);
   xpos = width/2;
   ypos = heigth/2;
   xspeed = 1;
  }









  void display(){
    rectMode(CENTER);
    fill(c);
    rect(xpos, ypos, 20, 10);
  }
  void drive(){
    xpos = xpos + xspeed;
    if(xpos > width){
      xpos = 0;
    }
  }
}
```

Tutorials Objects Cars

```
Car myCar;

void setup(){
  myCar = new Car();
}

void draw(){
  background(255);
  myCar.drive();
  myCar.display();
}
```

```
class Car {
  color c;
  float xpos;
  float ypos;
  float xspeed;

  Car(){
   c = color(255);
   xpos = width/2;
   ypos = heigth/2;
   xspeed = 1;
  }

  void display(){
    rectMode(CENTER);
    fill(c);
    rect(xpos, ypos, 20, 10);
  }
  void drive(){
    xpos = xpos + xspeed;
    if(xpos > width){
      xpos = 0;
    }
  }
}
```

Tutorials Objects Cars

```
Car myCar1;
Car myCar2;

void setup(){
  size(200, 200);
  myCar1 = new Car(color(255,0,0),0,100,2);
  myCar2 = new Car(color(0,0,255),0,10,1);
}

void draw(){
  background(255);
  myCar1.drive();
  myCar1.display();
  myCar2.drive();
  myCar2.display();
}
```

```
class Car {
  color c;
  float xpos;
  float ypos;
  float xspeed;

  Car(color tempC, float tempXpos, float tempYpos,
                                        float tempXspeed){
   c = tempC;
   xpos = tempXpos;
   ypos = tempYpos;
   xspeed = tempXspeed;
  }

  void display(){
    stroke(0);
    fill(c);
    rectMode(CENTER);
    rect(xpos, ypos, 20, 10);
  }

  void drive(){
    xpos = xpos + xspeed;
    if(xpos > width){
      xpos = 0;
    }
  }
}
```